

《软件体系结构》质量属性（Quality Attributes）完整知识点整理

课件来源：南京大学软件学院 2026UG SysArch 2-3 Quality Attributes

补充来源：EagleBear2002 博客笔记、《Software Architecture in Practice》第4版

关联考试：2026年期末考试 QI-1（需求/质量属性/ASR）、QI-3（质量属性场景建模）、QI-4（模式与战术）等多道题直接相关

 质量属性是软件架构设计的核心驱动因子。架构的本质上是为了实现特定的质量属性而做的设计决策集合。

目录

- [一、基本概念](#)
- [二、质量属性场景建模（核心考点）](#)
- [三、战术（Tactics）概念](#)
- [四、可用性（Availability）](#)
- [五、可修改性（Modifiability）](#)
- [六、性能（Performance）](#)
- [七、安全性（Security）](#)
- [八、可测试性（Testability）](#)
- [九、易用性（Usability）](#)
- [十、互操作性（Interoperability）](#)
- [十一、约束（Constraints）](#)
- [十二、七类设计决策与绑定时间](#)
- [十三、质量属性总结与考试要点](#)

一、基本概念

1.1 什么是软件架构？

《Software Architecture in Practice》定义：

软件架构是系统的一组结构（structures），包含软件元素（elements）、它们之间的关系（relations），以及两者的属性（properties）。

关键认知：

要点	说明
单独的盒式图不是架构	Box-and-line drawings alone are not architecture, but a starting point
架构包含组件的行为	Architecture includes behaviour of components
架构是最难更改的早期决策	An architecture represents those design decisions that are hardest to change
所有架构都是设计，但并非所有设计都是架构	Architecture is a subset of design

1.2 架构为什么重要？

- 1. 沟通工具：为涉众提供讨论需求、成本、进度的参考框架
- 2. 早期决策的体现：约束实现和开发者（Implementation must conform to architecture）
- 3. 组织结构的基础：决定团队划分、预算分配、工作拆分、集成、测试、运维
- 4. 促进/阻碍质量属性的实现：架构直接影响 modifiability、security、usability 等
- 5. 可迁移可重用的抽象：一对多映射（一种架构，许多系统）
- 6. 产品线通用性的基础：整个产品线共享一个架构（CBSE）

1.3 功能需求 vs 质量需求 vs 约束

概念	英文	定义	关键特征
功能需求	Functional Requirements	系统必须做什么，定义系统的行为	很大程度上与结构无关。功能可以通过许多不同的结构来实现
质量需求	Quality Requirements (Quality Attributes)	系统在功能之上应提供的可度量特性	不能在开发完成后"改装"上去，必须在每个设计决策中考虑
约束	Constraints	自由度为0的设计决策	预先已做出的、不可协商的设计决策

★ 核心认知：不可能先做对功能，再试图"适应"非功能性需求（NO retro-fitting quality）。质量属性必须在架构设计的每个决策中同时考虑。

1.4 质量属性的两大分类

分类	特征	例子
运行时可观察（外部）	系统运行时表现出来的行为特性	性能、安全性、可用性、易用性、可靠性、互操作性
开发时可观察（内部）	系统的静态结构特性，运行时不可见	可修改性、可移植性、可重用性、可测试性、可维护性

1.5 质量属性常见列表（23种）

质量属性	英文	分类	简要说明
性能	Performance	运行时	系统多快响应
可用性	Availability	运行时	系统什么时候能用
安全性	Security	运行时	谁能访问，数据是否安全
易用性	Usability	运行时	用户使用多容易
可靠性	Reliability	运行时	结果是否可以信赖
互操作性	Interoperability	运行时	能否和其他系统协作
响应能力	Responsiveness	运行时	响应速度
可恢复性	Recoverability	运行时	从故障中恢复的能力
可修改性	Modifiability	开发时	修改起来容易吗
可测试性	Testability	开发时	测试容易吗
可重用性	Reusability	开发时	能给别人用吗
可移植性	Portability	开发时	能跑在不同环境吗
可维护性	Maintainability	开发时	维护成本多高
可扩展性	Extensibility/Scalability	开发时	能扩展新功能/扩容吗
灵活性	Flexibility	开发时	适应变化的能力
模块化	Modularity	开发时	模块划分是否清晰
可配置性	Configurability	开发时	通过配置改变行为
可支持性	Supportability	开发时	运维支持多容易
可审核性	Auditability	开发时	行为的可追踪性
适应性	Adaptability	开发时	适应新环境的能力
稳定性	Stability	开发时	改动对系统影响小
可管理性	Manageability	开发时	系统管理多容易
可持续性	Sustainability	开发时	长期维护的可行性

★ **核心认知：**业务目标决定系统必须具备哪些质量属性。系统通常因为**缺乏所需的质量级别**（难以维护、移植或扩展）而被重新设计，而不是因为功能不够。

二、质量属性场景建模（核心考点）

★ **对应考试 Q1-3：**如何建模质量属性场景？用刺激-响应图画可用性和可修改性。

2.1 为什么需要精确建模？

质量属性不完全取决于设计，也不取决于实现或部署。要在架构级别对其进行评估，必须对质量属性进行**精确定义**。Quality attribute scenarios 就是这种精确定义的工具。

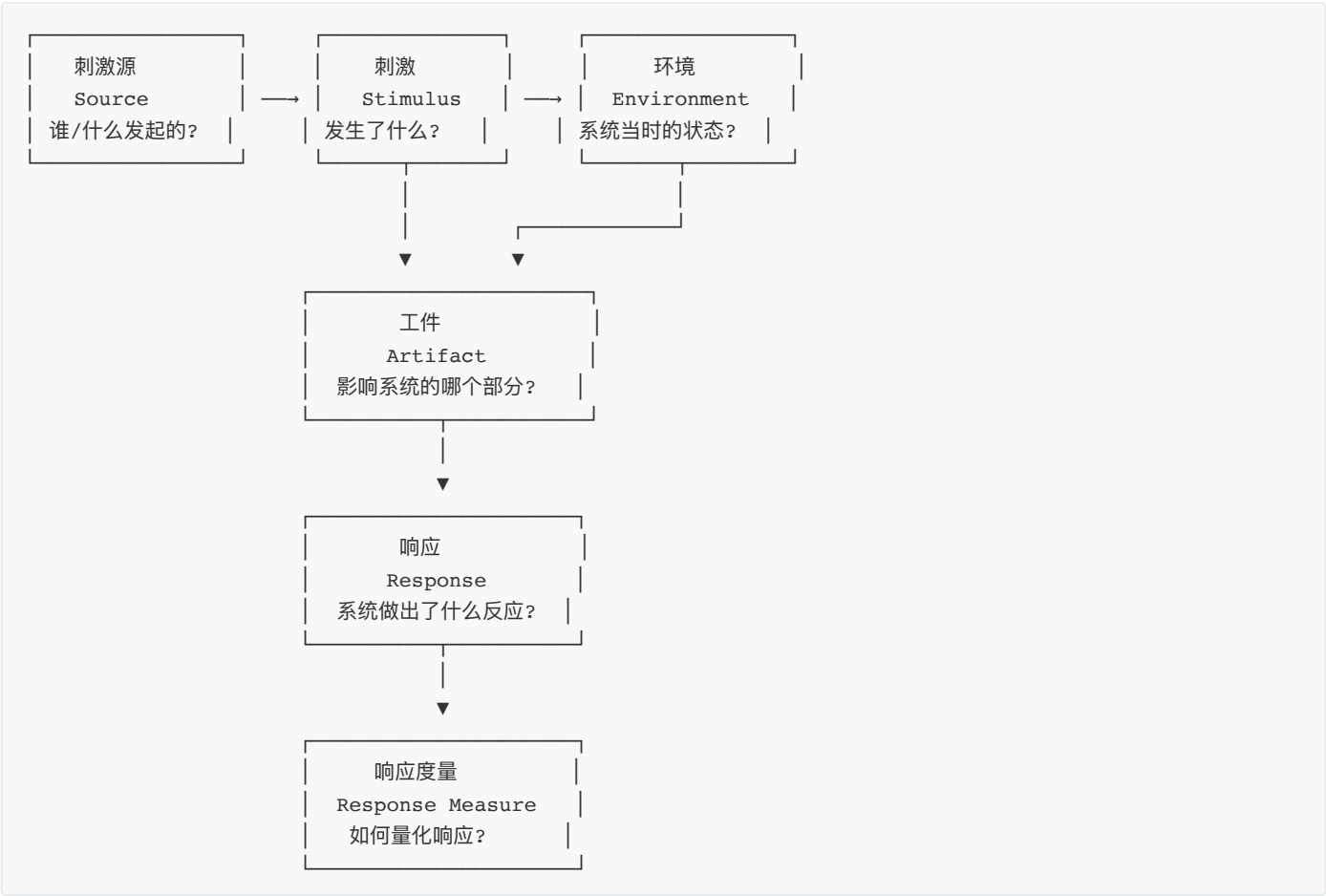
2.2 通用场景 vs 具体场景

类型	英文	特征
通用场景	General Scenarios	与系统无关，提供框架用于生成大量质量属性特定的场景
具体场景	Concrete Scenarios	系统特定的场景，是通用场景的具体实例化

这个场景就是 4+1 视图中的 "1"（Use Case/Scenario）。

2.3 质量属性场景的6要素

这是考试必考的核心框架。



#	要素	英文	要回答的问题	例子（可用性场景）
1	刺激源	Source	谁/什么产生刺激？	外部监控系统、用户、硬件故障
2	刺激	Stimulus	发生了什么？到达系统的条件是什么？	服务器心跳信号丢失、用户发起无效请求
3	环境	Environment	系统在什么状态下接收刺激？	正常运行、过载、遭受攻击、网络故障
4	工件	Artifact	系统的哪个部分受影响？	核心业务进程、通信通道、数据存储
5	响应	Response	系统做了什么？	切换到备用节点、记录日志、发送告警
6	响应度量	Response Measure	如何量化地衡量响应？	故障切换时间 < 30秒、可用性 ≥ 99.95%

🧠 记忆口诀：谁（Source）在什么状态（Environment）下发出了什么（Stimulus），影响了什么（Artifact），系统做了什么（Response），做得怎么样（Response Measure）。

三、战术（Tactics）概念

★ 对应考试 Q1-4：说明架构模式和战术的区别与关系。

3.1 什么是战术？

战术（Tactic）是影响质量属性响应控制的单一设计决策。

- 是一种原子级别的设计手段
- 战术的集合称为**架构策略（Architectural Strategy）**
- 战术可以像模式一样层层组合（例如冗余可以细分为数据冗余、计算冗余）

3.2 模式 vs 战术

维度	战术（Tactic）	模式（Pattern）
粒度	原子级，单一设计决策	组合级，一组设计决策的集合
关注点	通常影响一个质量属性	影响多个质量属性，做跨属性权衡
关系	是模式的组成部分	由多个战术构成
例子	冗余、缓存、加密	分层模式、微服务模式、事件驱动模式

风格/模式运用战术来提供预期的收益。Style or pattern applies tactics to provide the promised benefit.

四、可用性（Availability）

可用性是大多数IT应用程序的关键要求。

4.1 核心概念

定义

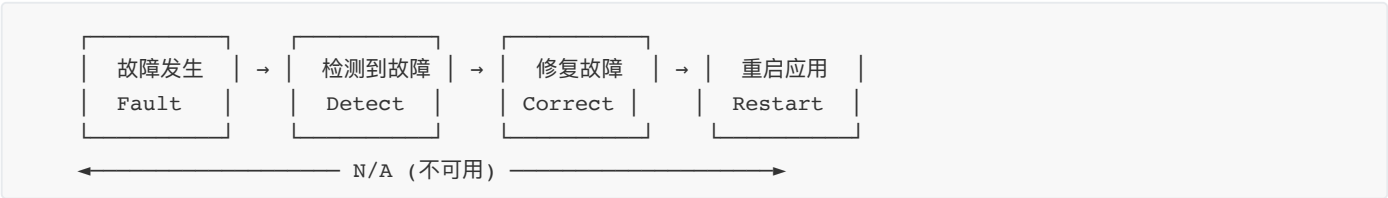
可用性是指系统在需要时能够**正常运行并提供服务**的能力。

可用性 vs 可靠性（重要区分）

概念	英文	含义
可用性	Availability	系统 可以用 ，但不保证结果一定正确
可靠性	Reliability	系统 持续正确运行、结果可信任

不可靠的应用可用性差，但可用的应用不一定可靠。

可用性损失的时间链



提高可用性的核心思路：尽可能缩短 N/A 的时间。

核心公式

Availability = MTBF / (MTBF + MTTR)

缩写	全称	含义
MTBF	Mean Time Between Failures	平均无故障时间 — 系统正常运行的平均时长
MTTR	Mean Time To Repair	平均修复时间 — 从故障到恢复的平均时长

计算可用性时，计划内的停机时间可能不计入。

关键术语辨析

术语	英文	含义
故障（原因）	Fault	系统中可能导致failure的潜在缺陷
错误（中间态）	Error	Fault发生后到Failure发生前的中间状态
失效（可观察）	Failure	系统无法交付预期服务的可观察状态
中断	Outage	系统不可用的情况（计划的停机也是一种Outage）

关系链：Fault → Error → Failure（Fault是原因，Failure是可观察的结果，Error是中间状态）

4.2 可用性战术

可用性战术

- 故障检测 (Fault Detection)
 - Ping/Echo — 发送探测包，检查响应
 - Monitor — 持续监控运行状态
 - Heartbeat — 定时心跳信号（死人和活人）
 - Timestamp — 时间戳校验（消息顺序是否正确）
 - Sanity Checking — 合理性/常识检查（数据是否符合常识）
 - Condition Monitoring — 条件监控（特定条件触发检查）
 - Voting — 投票机制（三模冗余，少数服从多数）
- 故障恢复-准备与修复 (Fault Recovery - Preparation & Repair)
 - Active Redundancy — 主动冗余（所有节点同时处理，切换最快，资源开销最大）
 - Passive Redundancy — 被动冗余（主节点处理，备节点待命，定期同步状态）
 - Spare — 冷备/备用（需要启动时间，资源开销最小）
- 故障恢复-重新集成 (Fault Recovery - Reintegration)
 - Shadow Mode — 影子模式（新节点先旁路观察再接管）
 - State Resynchronization — 状态重新同步
 - Escalating Restart — 逐级重启（先尝试最小恢复，失败再升级）

4.3 三种冗余方式对比

冗余方式	切换时间	资源开销	切换复杂度	适合场景
Active Redundancy（主动冗余）	最快（毫秒级）	最高（所有节点满负荷）	低（自动切换）	金融交易、电信核心
Passive Redundancy（被动冗余）	较快（秒级）	中等（备节点待机同步）	中（需状态同步）	Web服务器、应用服务器
Spare（冷备）	慢（分钟级）	最低（冷备资源池）	高（需启动配置）	非关键业务、批处理

4.4 可用性场景示例

要素	内容
刺激源	服务器硬件（Source：内部硬件故障）
刺激	主服务器宕机，心跳信号丢失（Stimulus：Failure detected）
环境	系统在正常负载下运行（Environment：Normal operation）
工件	核心订单处理进程（Artifact：Order processing process）
响应	系统检测到故障，自动切换到备用节点，通知运维（Response：Failover to standby）
响应度量	故障切换时间 < 30秒；服务可用性 ≥ 99.95%（Response Measure：30s, 99.95%）

五、可修改性（Modifiability）

5.1 核心概念

可修改性是指系统可以被经济高效地修改而不引入新错误的能力。

- 系统的 80% 工作量发生在部署之后
- 架构将变化分为三种类型：

变化类型	范围	影响
Local（局部）	单个组件修改	影响最小
Non-local（非局部）	几个组件修改	影响中等
Architectural（架构级）	修改系统基本结构、通信和协调机制	影响最大，代价最高

5.2 可修改性战术

可修改性战术

— 减小模块尺寸（Reduce Size of Module）

— Split Module — 将大模块拆分为更小的模块

— 增加内聚（Increase Cohesion）

└─ Semantic Cohesion	— 将语义相关的职责放在一起
└─ 减少耦合 (Reduce Coupling)	
└─ Encapsulation	— 封装内部实现细节
└─ Use an Intermediary	— 使用中介者/中间层解耦
└─ Abstract Common Services	— 抽象公共服务 (接口编程)
└─ Dependency Inversion	— 依赖倒置 (依赖抽象而非具体)
└─ Restrict Communication	— 限制通信路径
└─ 推迟绑定 (Defer Binding)	
└─ Runtime Registration	— 运行时注册/服务发现
└─ Configuration Files	— 配置文件驱动
└─ Polymorphism	— 多态替代条件判断
└─ Dependency Injection	— 依赖注入
└─ Component Replacement	— 组件可替换

5.3 可修改性场景示例

要素	内容
刺激源	产品经理 (Source: Product manager)
刺激	要求增加微信支付方式 (Stimulus: Add new payment method)
环境	系统已上线运行 (Environment: In production)
工件	支付模块 (Artifact: Payment module)
响应	修改支付接口的实现, 不影响其他模块 (Response: Localized modification)
响应度量	修改涉及 < 3个文件; 修改+测试 < 2人天; 无回归问题 (Response Measure: < 3 files, < 2 person-days)

六、性能 (Performance)

6.1 核心概念

性能是指系统在给定时间约束内响应事件的能力。

性能关注的核心指标：

- 延迟 (Latency)：单个请求从发出到收到响应的时间
- 吞吐量 (Throughput)：单位时间内能处理的请求数量
- 响应时间分布：不只是平均值，还包括尾部延迟 (P99、P999)

6.2 性能战术

性能战术	
└─ 资源需求控制 (Control Resource Demand)	
└─ Manage Sampling Rate	— 管理采样率 (降低处理频率)
└─ Limit Event Response	— 限制事件响应范围
└─ Prioritize Events	— 对事件进行优先级排序
└─ Reduce Overhead	— 减少开销 (精简处理流程)
└─ Bound Execution Times	— 限制执行时间
└─ Increase Efficiency	— 提高算法效率

—	资源管理 (Manage Resources)
—	— Increase Resources — 增加资源 (纵向扩展 Scale Up)
—	— Introduce Concurrency — 引入并发 (多线程/多进程)
—	— Maintain Multiple Copies — 维护多份副本 (横向扩展 Scale Out)
—	— Maintain Multiple Versions — 维护多版本 (金丝雀、蓝绿)
—	— Schedule Resources — 调度资源 (优先级调度)
—	资源仲裁 (Resource Arbitration)
—	— Fixed-Priority Scheduling — 固定优先级调度
—	— First-In/First-Out — 先来先服务
—	— Dynamic Priority — 动态优先级调度

6.3 性能场景示例

要素	内容
刺激源	大量并发用户 (Source: Users)
刺激	5万用户同时提交选课请求 (Stimulus: 50K concurrent requests)
环境	选课高峰期 (Environment: Peak enrollment period)
工件	选课系统 (Artifact: Course enrollment system)
响应	请求被处理, 返回确认或队列位置 (Response: Process requests with queuing)
响应度量	P99延迟 < 2秒; 吞吐量 > 1万/秒 (Response Measure: P99 < 2s, TPS > 10K)

七、安全性 (Security)

7.1 核心概念

安全性是指系统在保护信息和数据免受未经授权访问的同时，仍能为合法用户提供服务的程度。

安全性的 CIA 三元组：

要素	英文	含义
机密性	Confidentiality	数据不被未经授权访问
完整性	Integrity	数据不被未经授权修改
可用性	Availability	服务对合法用户可用 (防DDoS)

7.2 安全性战术

安全性战术
— 检测攻击 (Detect Attacks)
— — Intrusion Detection — 入侵检测系统
— — Detect Message Delay — 检测消息延迟 (中间人攻击)
— — Detect Service Denial — 检测拒绝服务攻击
— 抵抗攻击 (Resist Attacks)
— — Identify Actors — 识别行动者 (认证 Authentication)

├─ Authenticate Actors	— 验证身份
├─ Authorize Actors	— 授权访问 (Authorization)
├─ Limit Access	— 限制访问
├─ Limit Exposure	— 限制暴露面
├─ Encrypt Data	— 数据加密 (静态+传输中)
├─ Separate Entities	— 实体分离 (隔离敏感数据)
└─ Change Default Settings	— 修改默认设置
├─ 响应攻击 (React to Attacks)	
├─ Revoke Access	— 撤销访问权限
├─ Lock Computer	— 锁定系统
└─ Inform Actors	— 通知相关人员
└─ 从攻击中恢复 (Recover from Attacks)	
├─ Audit	— 审计 (追踪发生了什么)
└─ Maintain Audit Trail	— 维护审计日志 (不可篡改)

7.3 安全性场景示例

要素	内容
刺激源	恶意攻击者 (Source: Malicious attacker)
刺激	尝试通过SQL注入窃取用户数据 (Stimulus: SQL injection attempt)
环境	系统在线运行 (Environment: Online)
工件	用户数据库 (Artifact: User database)
响应	防火墙检测到异常模式, 阻止请求, 记录审计日志, 通知安全团队 (Response: Block & log)
响应度量	攻击被阻止时间 < 1ms; 审计日志完整记录; 无数据泄露 (Response Measure: < 1ms block, no data leak)

八、可测试性 (Testability)

8.1 核心概念

可测试性是指通过测试来揭示软件缺陷的容易程度。

8.2 可测试性战术

```

可测试性战术
├─ 控制和观察系统状态 (Control and Observe System State)
│   ├── Specialized Interfaces — 专门的测试接口
│   ├── Record/Playback — 录制/回放
│   ├── Localize State Storage — 状态存储局部化
│   ├── Abstract Data Sources — 抽象数据源 (可替换测试数据)
│   ├── Sandbox — 沙箱隔离测试
│   └── Executable Assertions — 可执行断言
└─ 限制复杂度 (Limit Complexity)
    ├── Limit Structural Complexity — 限制结构复杂度 (避免深度嵌套)
    └── Limit Non-determinism — 限制非确定性 (时间、随机、并发)
  
```

九、易用性（Usability）

9.1 核心概念

易用性是指用户使用系统完成任务的难易程度，以及系统的支持程度。

9.2 易用性战术

易用性战术

支持用户主动行为 (Support User Initiative)

Cancel

Undo

Pause/Resume

Aggregate

Multiple Views

取消操作

撤销操作

暂停/恢复

聚合信息展示

多重视图

支持系统主动行为 (Support System Initiative)

Maintain Task Model

Maintain User Model

Maintain System Model

维护任务模型 (系统理解用户在做什么)

维护用户模型 (系统了解用户偏好)

维护系统模型 (系统了解自己的状态)

提供反馈 (Provide Feedback)

Progress Indication

System State

Confirmation

进度指示

系统状态显示

确认提示

十、互操作性（Interoperability）

10.1 核心概念

互操作性是指两个或更多系统通过接口交换和利用信息进行协作的程度。

10.2 互操作性战术

互操作性战术

定位 (Locate)

Discover Service

服务发现机制

管理接口 (Manage Interfaces)

Orchestrate

Tailor Interface

Standardize Interface

编排 (中央控制协调多方交互)

定制接口 (适配不同消费者需求)

标准化接口

数据交换 (Data Exchange)

Transform Data Format

Ensure Data Integrity

数据格式转换

确保数据完整性

十一、约束（Constraints）

11.1 核心概念

约束是具有零自由度的设计决策。A constraint is a design decision with ZERO degrees of freedom.

- 约束是已经做出的、预先指定的设计决策
- 约束限定了架构的搜索空间，架构是在这个边界内找最优解
- 满足约束 = 接受这个设计决策 + 与其他受影响的设计决策协调

11.2 约束的三种类型

类型	说明	例子
时间与预算	项目截止日期和财务限制	3个月交付 → 不能用需要6个月开发的技术
技术限制	团队能力和现有技术栈的限制	团队不懂区块链 → 不加区块链子需求
业务规则与法规	必须遵守的外部规则	电商系统必须遵守GDPR

十二、七类设计决策与绑定时间

★ 对应考试 QII-1（10分论述题）：简述七类设计决定，说明绑定时间有哪些选择，分析不同绑定时间影响哪些质量属性。

12.1 七类设计决策

Architecture is a collection of design decisions. 架构就是设计决策的集合。

#	类别	英文	核心问题	例子
1	职责分配	Allocation of Responsibilities	谁负责什么？	认证交给Auth Service，订单交给Order Service
2	协调模型	Coordination Model	各部分怎么沟通？	同步REST调用 vs 异步消息 vs RPC
3	数据模型	Data Model	数据怎么组织？	关系型 vs NoSQL；每服务独立DB vs 共享DB
4	资源管理	Management of Resources	CPU/内存/网络怎么分配？	连接池、缓存策略、限流、内存管理
5	架构元素间的映射	Mapping Among Architecture Elements	架构怎么落实到代码/团队/硬件？	模块 → 代码包；服务 → 容器；模块 → 开发团队
6	绑定时间决策	Binding Time Decisions	变化什么时候被固定下来？	编译时绑定 vs 部署时绑定 vs 运行时绑定
7	技术选择	Choice of Technology	用什么具体技术？	Spring Boot vs Go；MySQL vs PostgreSQL

🧠 决策的顺序逻辑：一般先做职责分配（拆开）→ 协调模型（怎么通信）→ 数据模型（数据怎么管）→ 资源管理 → 映射 → 绑定时间 → 最后才是技术选择。如果一上来就选技术栈，是**本末倒置**——没搞清楚系统需要什么。

12.2 绑定时间（Binding Time）

绑定时间：系统中的**一个可变决策可以在什么时间点之前被确定**。在这个时间点之前系统保持灵活性，之后便不可更改。

早绑定（灵活性低，确定性高，性能好）
|



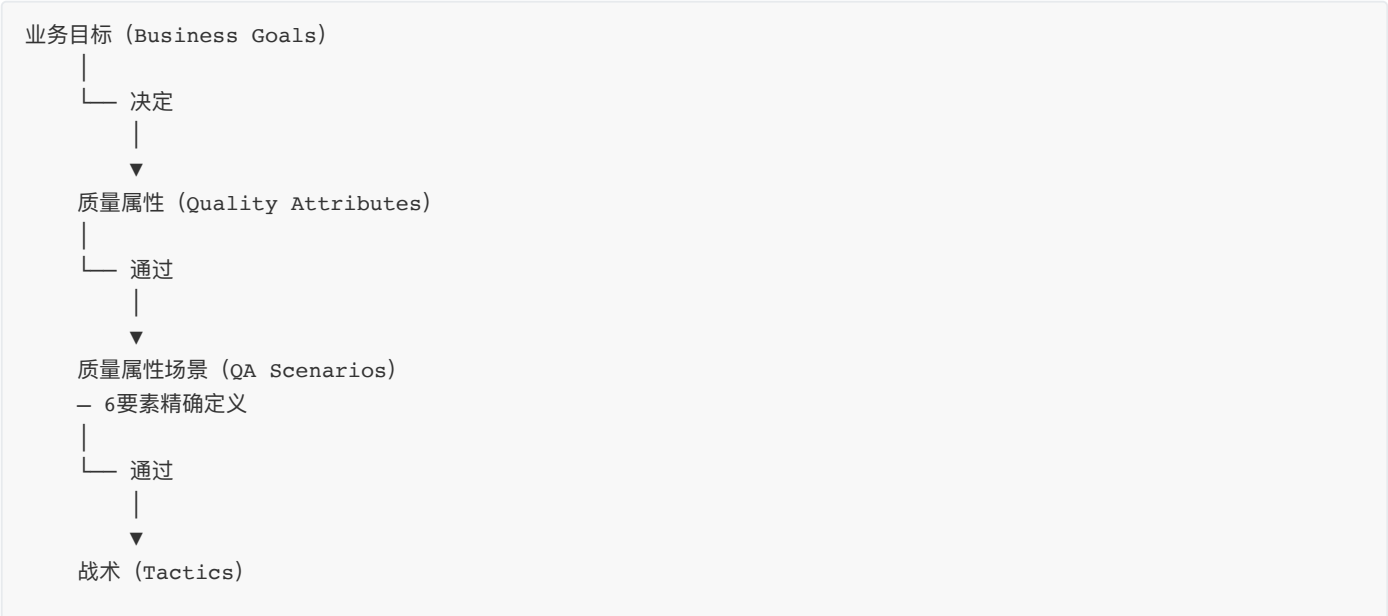
12.3 绑定时间对质量属性的影响

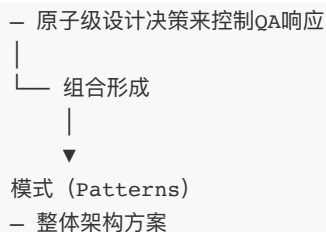
绑定时间	可修改性	性能	可部署性	可测试性	可理解性
编码时	↓↓	↑↑	↓↓	↓	↑↑
编译时	↓	↑↑	↓	↓	↑
部署时	↑	↑	↑↑	↑	↑
运行时	↑↑	↓	↑↑	↓↓	↓

 **权衡核心：**越晚绑定 → 灵活性越高 → 可修改性越好，但性能越差、越难测试。部署时绑定是大多数场景的最佳平衡点。运行时绑定虽然最灵活，但也引入了最大的不确定性和排障难度。

十三、质量属性总结与考试要点

13.1 质量属性的层级关系





13.2 七大质量属性战术速查

质量属性	战术大类	核心思路
可用性	检测故障 + 恢复故障	让不可用时间尽量短
可修改性	减小模块 + 增加内聚 + 减少耦合 + 推迟绑定	把变化影响锁定在最小边界内
性能	控制需求 + 管理资源 + 资源仲裁	用更少的资源做更多的事
安全性	检测 + 抵抗 + 响应 + 恢复	纵深防御，不让一个人突破整条防线
可测试性	控制观察 + 限制复杂度	让缺陷无处可藏
易用性	支持用户 + 系统主动 + 反馈	让用户更高效地完成任务
互操作性	定位 + 管理接口 + 数据交换	让不同系统说同一种语言

13.3 核心认知归纳

- 1. 质量属性不能后加：不可能先做对功能再"改装"质量属性 (NO retro-fitting quality)
- 2. 质量属性彼此拉扯：性能、可维护性、安全性、可扩展性常常不能同时最优，架构选择本质是排序
- 3. 质量属性场景是桥梁：将模糊的"系统要快"转化为可量化的"P99延迟 < 2秒"
- 4. 战术是原子操作：每个战术只影响1-2个质量属性，模式是战术的组合
- 5. 架构就是设计决策的集合：七大设计决策类型 + 绑定时间是核心分析工具
- 6. 约束是自由度为0的决策：架构是在约束边界内找到最优解



延伸阅读：

- *Software Architecture in Practice* (4th ed.) — Bass, Clements, Kazman
- [EagleBear2002 质量属性笔记](#)



关联文档：

- 《软件架构模式-完整知识点整理.md》 — 9个时代架构演进与质量属性取舍
- 《ADD 3.0 设计方法-完整知识点整理.md》 — 如何使用战术和模式进行系统化架构设计
- 《软件体系结构-期末考试高分复习指南.md》 — 全题型覆盖的考试答题指南

整理自：2026UG SysArch Quality Attributes 课件 + EagleBear2002 课程笔记 + SAiP 第4版